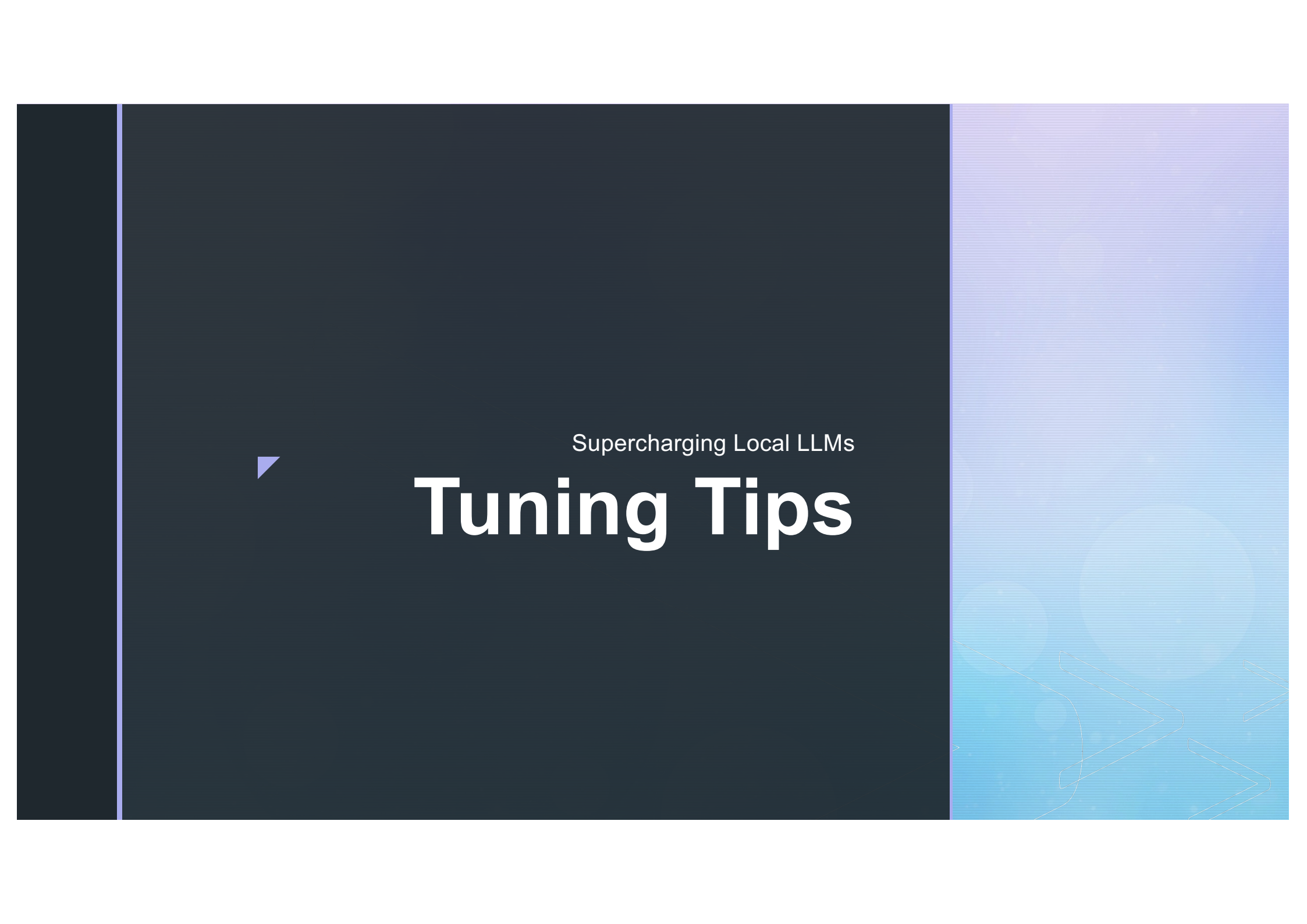


OWN YOUR AI

RUN · LOCAL · SOVEREIGN



Supercharging Local LLMs

Tuning Tips



Andreas Burner

Management Advisor for AI & Cloud Strategy, Court-Certified Expert

Working 25+ years in enterprise technology

Active since 2000 with ML / AI

Current projects

Sovereign AI for 8M+ Austrians

SITU Agent Open Source – “Nothing leaves the room” (situagent.com)

WHY TUNING ?

Defaults are built for the cloud, not your box

generic settings waste your VRAM and silently truncate output

Local means you own every knob

KV-cache quant, batch size, speculative decoding ...

Tuning them is the gap between 8 and 40 tokens/sec.

Tuning closes the gap to frontier models

The right settings make small models better, or let you run large ones

SPEED · QUALITY · STABILITY

TUNING WHAT?

Harness / Agent
(Claude, PI, OpenCode, etc.)

LLM Inference Engine
(Llama.cpp, Olama, vLLM, etc.)

Max Tokens

→ harness config

WHAT ?

- *max_tokens* = how many tokens the model is **allowed to write** in one reply
- **!= context size**
context is (full history + new prompt + reply); *max_tokens* only is the reply
- Engines (llama.cpp, Ollama) **don't cap by default**
the agent frontend/harness does, usually at 4096

WHY ?

- Reasoning models **eat the budget invisibly** - hidden thinking tokens compete with output tokens
- When hitting the cap **stops mid-sentence** - no error, no warning
- Complex tasks often **exceed 10,000 tokens**; even with agent loops initial pass often blows 4096
- Avoids agent retry loops (3-5 LLM calls)
- Real GitHub PRs: real-world fixes consume **8K+ output tokens**

Max Tokens

→ harness config

HOW ?

- Set to **16384**. This is 4× the legacy 4096, 2× the modern 8192 baseline
- **Zero memory cost.** KV cache is driven by `--ctx-size`, not this

Context Size

→ LLM config

WHAT ?

- KV cache = **remembered attention state** per token (avoids recomputing the past)
- --ctx-size sets **KV cache capacity** in tokens (~160 KB/token on a 14B model)
- **Hard cap (full history + new prompt + reply)** - Exceed: compact, truncate, or recompute
- Every GGUF embeds n_ctx_train - the value the model was **actually trained for**
- Read it from the model **instead of hardcoding** a number per machine

HOW ?

- Query /v1/models → meta.n_ctx_train after server start
- **Do NOT purely rely** on --ctx-size 0

Context Size

→ LLM config

HOW ?

- **Too small** → previous turn's cache gets thrown out, every turn pays a full prefill
- **Too big** → RAM is reserved up front for the full window, even when empty
- Alibaba ships Qwen3 with a **1M-token** context - allocating it all for a 14B model would need ~160 GB of KV cache alone
- Over-allocating KV pushes layers off the GPU → CPU bottleneck → **~32% slower prefill**

What's new and coming?

- Ollama 2026 desktop GUI **exposes context length per-model**
- LM Studio **reads training context** from model cards automatically

Parallel slots

→ LLM config

WHAT ?

- Parallel slots = **how many simultaneous conversations** the server pre-reserves a KV cache for
- **Defaults vary across builds** (often 4, 1, auto) - check your own startup log, don't trust the docs

WHY ?

- Engines **allocate KV for EVERY slots at startup** - Qwen3-14B at ctx=180K is 21.7 GB per slot → 4 slots = 86.8 GB reserved before the first token
- 4 slots → 86.8 GB KV → **exceeds a 64 GB box** → disk swap → 0.34 tok/s
- 1 slot → 21.7 GB KV → **fits in RAM** → 3–6 tok/s

Parallel slots

→ LLM config

HOW ?

- **Read the startup log** or verify via API in llama.cpp /props → total_slots
- **Set to 1:** --parallel 1 (llama.cpp) or OLLAMA_NUM_PARALLEL=1 (Ollama)

What's new and coming?

- SGLang **RadixAttention**: share KV cache *across* requests
- For multi-user use **vLLM**, not llama.cpp

ContextWindow Sync

→ LLM config, harness config

WHAT ?

- **Two independent context counters:** the agent (Aider, OpenHands, Continue, PI, ...) tracks what it sends; the engine (llama.cpp) enforces `--ctx-size`
- They **never talk to each other** - and when they disagree, things break silently or loudly

WHY ?

- **Too high** → server rejects: *"request (66K tokens) exceeds available context (65K)"*
- **Too low** → agent compacts too early, throws away useful history before it had to
- Problem **"the agent forgot what we just decided"**

ContextWindow Sync

→ LLM config, harness config

HOW ?

- Tell the agent **exactly what the engine offers**:
 - Single-user: `contextWindow = ctx_size`
 - Multi-user: `contextWindow = ctx_size / parallel_slots` (llama.cpp splits the window across slots)
- **Derive both from one source** - never maintain two configs by hand

What's new and coming?

- Summaries live on disk, not in the context window – therefore context windows keep small

KV Cache Quantization

→ LLM config

WHAT ?

- KV cache = **remembered attention state** per token (avoids recomputing the past)
- Stores K and V vectors **2 bytes per token, per layer, per head** - for Qwen3-14B that's ~160 KB/token
- Quantizing = **downsampling bit width** - F16 → Q8 = half the bytes per element
- Q8 (1 byte) **halves it to ~10.9 GB** with almost same quality

WHY ?

- Qwen3-14B at ctx=180K → **21.7 GB KV in F16**, bigger than the model weights themselves (~8 GB Q4_0)
- Makes **large-context CPU inference feasible** on 32 GB boxes
- CPU throughput **improves** - bandwidth saving beats dequant cost

KV Cache Quantization

→ LLM config

HOW ?

- `--cache-type-k q8_0 --cache-type-v q8_0`
- **GPU:** keep F16 if VRAM allows, since Q8_0 may drift
- **Never Q4_0** - unpacking 4-bit values on every attention step is a massive overhead

What's new and coming?

- A lot: **TurboQuant** ICLR 2026 linear-state layers, no cache (Qwen3-Next)
 - Another talk ?

Pin the KV Cache in RAM

→ LLM config

WHAT ?

- OS silently **move "cold" memory** out from under the LLM
- macOS **compresses** heap pages; Linux **swaps** to disk; Windows **trims** the working set
- Between tokens the KV cache **sits idle** - the OS assumes it's unused and swaps it out
- "Machine feels fast for 2 min then dies" - that's this

WHY ?

- **Page fault mid-processing** = order-of-magnitude slowdown
- macOS M2 Max without tuning: 20+ tok/s → <2 tok/s (**10× regression**)
- Hits hardest on **long coding sessions** - bigger cache, more for the OS to swap out

Pin the KV Cache in RAM

→ LLM config

HOW ?

- Parameter mlock - tells the OS "**do not move these pages**", pinning the cache in RAM (works on all three OSes)
- Linux: also set vm.swappiness=0, consider huge pages
- Safe when **(model + KV + 4 GB) < 70%** of system RAM

What's new and coming?

- M5 Pro / M5 Max MacBook Pro (March 2026) - up to **128 GB** unified memory at 614 GB/s, headroom to safely pin larger models
- LM Studio exposes a "**Keep entire model in RAM**" (use_mlock) toggle in model-load settings

Reasoning Budget

→ LLM config

WHAT ?

- Reasoning is good for multi-step problems (math derivations, algorithm design, etc.)
- Not beneficial for lookups, formatting, single-step transforms
- Thinking models (Qwen3, Gemma-4) reason inside <think> blocks

WHY ?

- Thinking tokens **eat the generation budget**
- **Uncapped thinking** → no budget left for actual output
- Handoff message is **not optional** - without it you do worse than no thinking

Reasoning Budget

→ LLM config

HOW ?

- Current best practice: 25% reasoning, 75% output
- `--reasoning-budget $((MAX_TOKENS * 25 / 100))`
- Always pair with `--reasoning-budget-message` "Let me now write the solution."

What's new and coming?

- Per-call toggles in local engines
- Mid-stream budget signaling - periodically "X tokens left" so it can wind down on its own (BudgetThinker, arXiv 2508.17196)

Speculative Decoding

More Tokens/Sec, Same Output

→ LLM config

WHAT ?

- A small "draft" model cheaply guesses 3–5 tokens ahead
- The big model verifies them all in one pass
- Same output as normal decoding - pure speedup, no quality trade-off

WHY ?

- **1.5–3× tokens/sec** on coding workloads (very predictable)
- **No quality degradation**

Speculative Decoding

More Tokens/Sec, Same Output

→ LLM config

HOW ?

- Classic: `--model-draft draft-model.gguf` - load a small model alongside the main one
- Fit requirement: draft must share tokenizer and prediction style - otherwise predictions are rejected
- Modern: MTP (Multi-Token Prediction) - extra prediction heads trained into model, no separate file
- Cost: ~600 MB extra RAM for a draft model; zero for MTP

What's new and coming?

- MTP is standard in recent releases of DeepSeek-V3, Qwen3-Next, etc. prediction heads in model
- Gemma 4 ships with official paired MTP drafter checkpoints

Temperature

→ LLM config

WHAT ?

- Controls **how random the next-token pick** is (higher = more randomness)
- Engine defaults are **0.7 to 1.0** (llama.cpp/Ollama 0.8, vLLM 1.0) - often chat-tuned, not for code
- Tuning: **reproducibility vs. creativity**

WHY ?

- Good practice: set it manually - tools do not consider the model card
- Improves output quality significantly
- T=0.1 aligns runs with benchmarks
- At T=0.8 the same prompt diverges 30–40% of the time

Temperature

→ LLM config

HOW ?

- Respect the model's card (Qwen3 = 0.6, Gemma-4 = 1.0)
- Differs for thinking and non-thinking mode
- Benchmarking: --temp 0.1 (reproducible)
- Too low (0.0) causes repetition collapse

What's new and coming?

- Later in 2026: embedding sampling parameters inside the GGUF

WHY TUNING ?

SPEED · QUALITY · STABILITY



Andreas Burner